

COMPLETE BEGINNER'S GUIDE

Recruiter AI

How this project was built, and how every part of the code works — explained from zero.

Repo: github.com/Harbringer904/agent_max · Stack: Node.js + Express + vanilla JS

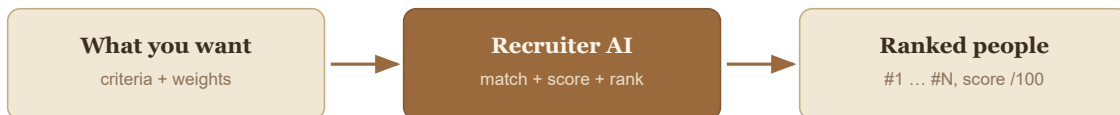
What this guide assumes: nothing. If you have never written code, you can still read this top-to-bottom and understand the whole project. Terms are defined the first time they appear, and again in the Glossary at the end.

How to read it

- Every section is short bullets + a diagram, not walls of text.
- Boxes like the one above are "plain-English" notes.
- Grey code blocks show the *actual* code or data from the project.
- The **Glossary** (last page) defines every technical word in one line.

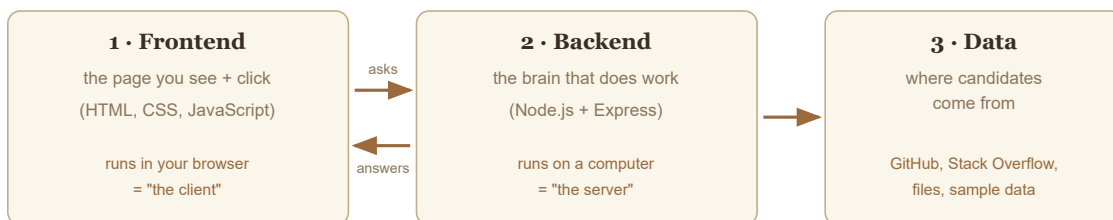
1. What we built (in one breath)

- A **website** where a recruiter types what they want in a candidate (skills, years of experience, certifications, etc.), and the app returns a **ranked list of people**, each with a **score out of 100** and a **rating out of 10**.
- It can pull candidates from **4 different sources** (GitHub, Stack Overflow, bundled sample data, or a file you upload).
- It works for **any job field** — software, nursing, finance, sales, marketing, design, operations — not just programmers.



2. Web-app foundations (skip if you know these)

- A **website** is made of three parts that talk to each other:



- **Client** = your browser (the thing showing the page). **Server** = a program running elsewhere that the client talks to.
- **API** = the "menu" of requests the server accepts. The client orders from the menu; the server cooks and returns a result. (Our menu items start with `/api/...`.)
- **JSON** = the plain-text format used to pass data between client and server. It looks like `{ "name": "Ada", "skills": ["python"] }`.

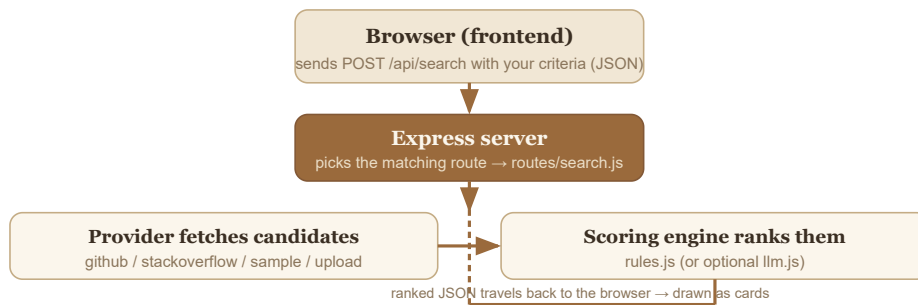
3. The tech stack — what each tool is, and why we used it

Tool	What it is (one line)	Why it's here
JavaScript	The programming language of the web.	The whole app is written in it — front and back.
Node.js	Lets JavaScript run <i>outside</i> a browser (on a server).	Powers the backend.
Express	A small Node library for building web servers + APIs.	Handles incoming requests and routes them.
HTML	The language that defines page <i>structure</i> (buttons, text).	The user interface skeleton.
CSS	The language that defines page <i>looks</i> (colors, spacing).	The warm "woody" theme.
JSON	A text format for structured data.	How data moves client ↔ server, and how sample candidates are stored.
npm	Node's "app store" for code libraries + a task runner.	Installs Express/dotenv; runs <code>npm test</code> .
REST API	A style of API using URLs + verbs (GET/POST).	Our <code>/api/...</code> endpoints.
Git	Tracks every change to the code (a time machine).	Version control.
GitHub	A website that hosts Git projects online.	Where the code is published.
LLM	"Large Language Model" — an AI like Claude/Gemini.	<i>Optional</i> human-readable reasoning for each candidate.
.env file	A private file holding secrets (API keys).	Keeps keys out of the code and out of GitHub.

Key fact: the app uses only **two** installed libraries — `express` (web server) and `dotenv` (reads the `.env` file). Everything else is built from scratch. Fewer libraries = less to break and less to learn.

4. The big picture — how a request flows

- When you click **Search**, this exact chain happens:



- The **folder structure** mirrors those jobs — each folder has one responsibility:

```

recruiter-ai/
├─ server.js           ← starts the server, connects the routes
├─ routes/             ← one file per API endpoint (the "menu")
│  ├─ health.js        ← GET /api/health   (is it alive? which AI is on?)
│  ├─ fields.js        ← GET /api/fields   (list of job fields)
│  ├─ templates.js     ← GET /api/templates (11 ready-made job specs)
│  ├─ search.js        ← POST /api/search  (the main event)
│  └─ results.js       ← save/reopen a result set (shareable links)
├─ lib/                ← the "brains" (no web stuff in here)
│  ├─ normalize.js     ← forces every candidate into one common shape
│  ├─ jobTemplates.js  ← the 11 pre-built job templates
│  └─ scoring/
│     ├─ rules.js      ← the math that turns a match into a 0-100 score
│     └─ llm.js        ← optional AI scoring + reasoning (with fallback)
├─ providers/          ← the 4 data sources, one file each
│  ├─ index.js         ← the registry ("which providers exist")
│  ├─ github.js        └─ stackoverflow.js
│  └─ sample.js        └─ upload.js
├─ data/samples/*.json ← bundled fake candidates per field
├─ public/index.html   ← the entire frontend (one file)
└─ test/*.test.js     ← 68 automated checks
  
```

5. The core idea — turn "requirements" into a score

- Everything revolves around **4 data shapes**. A "shape" just means: *these fields, in this arrangement*.

① **Candidate** — one person, in a standard form (every source is converted to this):

```
{ "name": "Rachel Thompson", "field": "healthcare",  
  "yearsExperience": 6, "educationLevel": 3,  
  "skills": ["patient care", "EHR"],  
  "certifications": ["RN", "BLS", "ACLS"],  
  "location": "Denver, CO", "summary": "..." }
```

② **JobSpec** — what the recruiter wants:

```
{ "field": "healthcare", "title": "Registered Nurse",  
  "criteria": [ ...a list of Criterion... ] }
```

③ **Criterion** — one requirement, with a **weight** (importance, 1–5):

```
{ "key": "certifications", "label": "Certifications",  
  "weight": 5, "requiredCerts": ["RN", "BLS"] }
```

④ **Scored candidate** — a Candidate + the numbers the engine adds:

```
{ "...": "... (all candidate fields) ...",  
  "criteriaScores": { "certifications": { "raw": 1, "weighted": 5, "max": 5, "note":  
    "matched 2/2" } },  
  "totalScore": 100, "rank1to10": 10 }
```

- There are exactly **6 kinds of criterion**. Each produces a **raw** match between **0** (no match) and **1** (perfect):

key	What it measures	How raw (0–1) is decided
skills	Required skills present	(matched skills) ÷ (required skills)
experience	Years of experience	candidate years ÷ required years (capped at 1)
education	Highest degree	meets required level → 1, else proportional
certifications	Required certificates held	(matched certs) ÷ (required certs)
location	Location match	matches → 1, else 0
keyword	Words in their summary/skills	(matched keywords) ÷ (total keywords)

- Education is a number ladder: 0 none · 1 high-school · 2 associate · 3 bachelor · 4 master · 5 doctorate .

6. The scoring engine — the math, made simple

- For **each** criterion: **weighted** = raw × weight , and **max** = weight .
- Add them up and turn into a percentage: **totalScore** = (sum of weighted ÷ sum of max) × 100 .

- The 1–10 rating is just `round(totalScore ÷ 10)` (never below 1).

Worked example — a nurse vs. the "Registered Nurse" job

criterion raw × weight = weighted (max)

certifications $1.0 \times 5 = 5.0$ (5)

education $1.0 \times 3 = 3.0$ (3)

experience $1.0 \times 3 = 3.0$ (3)

skills $1.0 \times 3 = 3.0$ (3)

sum weighted = 14

sum max = 14

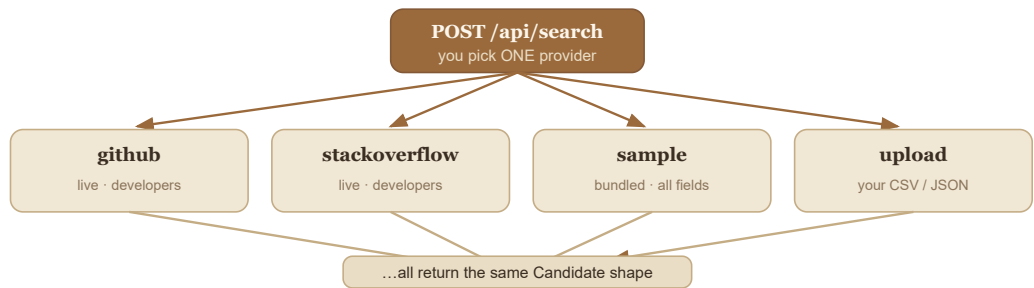
$$(14 \div 14) \times 100 = \text{totalScore } 100$$

$$\text{rating} = \text{round}(100 \div 10) = 10 / 10$$

- **Two beginner-friendly guarantees baked into the code:**
 - *Missing data never crashes it.* No education info → that criterion scores 0 with a note "no education data", not an error.
 - *Smart word matching.* "go" does **not** match inside "google", but "c++" and "node.js" match as whole words. (This lives in a helper called `skillMatches`.)
- The heavy lifting is ~160 lines in `lib/scoring/rules.js`, and it is **pure**: same inputs always give the same outputs, no internet, no surprises. That is exactly why it is easy to test.

7. Where candidates come from — the 4 "providers"

- A **provider** is a plug-in data source. All four return the same **Candidate** shape, so the rest of the app doesn't care which one was used.



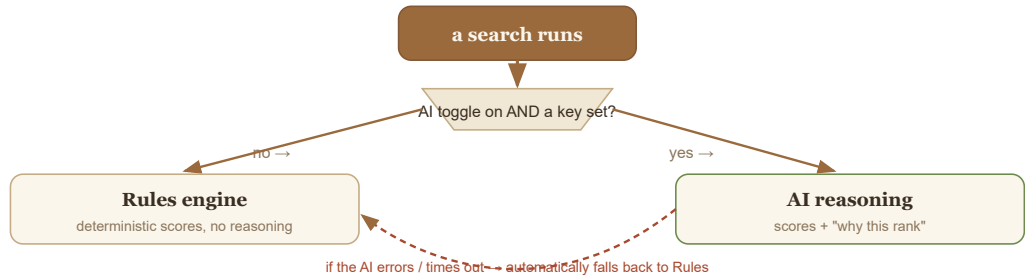
Provider	Source	Fields it serves	Needs a key?
github	GitHub's public API (real developers)	software	Optional (higher limits)
stackoverflow	Stack Exchange API (top answerers)	software	No
sample	Bundled JSON files in data/samples/	all fields	No
upload	A CSV or JSON you paste in	all fields	No

- Adding a *new* source later = write one new file that returns Candidates, then list it in `providers/index.js`. Nothing else changes. This is called a **pluggable architecture**.

8. The optional AI layer (and why it can never break the app)

- By default, scoring is 100% the **rules** math from Section 6 — free, instant, no AI.
- If you turn on the  **AI reasoning** toggle, an LLM adds a plain-English "*why this rank*" for each person.
- Hybrid design:** the AI only judges each criterion's raw 0–1 fit and writes the sentence. The *same weight math* is applied by our code — so AI scores and rule scores stay comparable.
- It supports **3 interchangeable AI providers**, auto-chosen by which key you set:

Env key you set	AI used	Cost
GROQ_API_KEY	Llama 3.3 (Groq)	Free
GEMINI_API_KEY	Gemini 2.0 Flash	Free
ANTHROPIC_API_KEY	Claude	Paid



- The key line: `rankCandidatesLLM(...)` is written to **never throw an error**. Missing key, bad network, timeout, weird response → it silently returns the rules-based result. The app cannot break because AI is unavailable.

9. The backend code, file by file

- `server.js` — the on-switch. Starts Express, serves the `public/` folder, connects the 5 route files, listens on port 3000. ~45 lines, pure wiring.
- `routes/health.js` — answers "am I alive, is a GitHub token set, which AI is active?"
- `routes/fields.js` — lists the job fields and which providers serve each.
- `routes/templates.js` — returns the 11 ready-made job templates.
- `routes/search.js` — the main endpoint. Checks your input is valid → asks the chosen provider for candidates → ranks them → applies any "Required" filters → returns the top results.
- `routes/results.js` — saves a finished result set to a file and hands back a short link id; another request reopens it. Powers **shareable links**.
- `lib/normalize.js` — the "translator". Turns any raw person (a GitHub user, a CSV row...) into the one standard **Candidate** shape, filling blanks safely.
- `lib/jobTemplates.js` — the 11 pre-filled job specs so you don't start from a blank form:

Field	Templates
software	Frontend Engineer · Backend Engineer · Data Scientist
healthcare	Registered Nurse · Medical Lab Technician
finance	Accountant · Financial Analyst
sales / marketing	Sales Representative · Marketing Manager
design / operations	UX Designer · Project Manager

- `lib/scoring/rules.js` — the free, deterministic scoring math (Section 6).
- `lib/scoring/llm.js` — the optional AI scoring + fallback (Section 8).
- `lib/providers/*` — the 4 data sources + `index.js` registry (Section 7).
- `lib/resultsStore.js` — reads/writes the saved result files, with a safety check on the id so a sneaky link can't read other files on the computer (**path-traversal protection**).

10. The frontend — one HTML file

- The entire user interface is a single self-contained file: `public/index.html` (structure + styles + logic together).
- No frameworks (no React/Vue) — just **vanilla JavaScript**. Easier to read as a beginner: what you see is what runs.
- The screen flow:

Pick a field → pick a template → tweak the criteria + weights
→ choose a data source → Search → see ranked candidate cards

- Built-in recruiter tools on the results: ★ **shortlist**, 🗑 **compare** (side-by-side table), **CSV / PDF export**, 🕒 **history**, ★ **saved searches**, 🔗 **shareable link**, and a "Required" checkbox that hard-filters must-haves.
- **Remembering things:** shortlist, history, and saved searches live in your browser's **localStorage** (a tiny per-browser storage) — no login, no database.
- The **"woody" theme** is pure CSS variables, so the whole look is controlled from ~15 values at the top:

```
--bg:#EFE6D6;      /* warm cream page */  --surface:#FBF6EC; /* ivory cards */
--accent:#9A6A3C; /* walnut buttons */  --text:#3A2E22;    /* espresso text */
```

```
--green:#6E8B4E; /* high score */ --border:#D8C6A8; /* soft tan */
```

11. Security & quality (why it's safe to run and share)

- **Secrets stay secret.** Real keys live only in a `.env` file, which is listed in `.gitignore` so it is **never** uploaded to GitHub. The public repo only has `.env.example` with blank placeholders.
- **Errors don't leak internals.** Every API error is a short opaque code, never a stack trace:

Code	Meaning	HTTP
R001	Invalid job spec	400
R002	Unknown provider	400
R003	Candidates missing/empty	400
R004	Saved result not found	404
R500	Unexpected internal error	500

- **68 automated tests** (`npm test`) check the scoring math, the data translator, the CSV parser, and the result store — so a future change that breaks something is caught instantly.
- **Input validation** rejects malformed requests before they reach the logic.
- **Honesty:** a `FAIRNESS.md` states this is *decision-support*, not an autopilot — a human should always review. Scores are heuristics, and signals like "years on GitHub" are rough proxies.

12. How this was actually built — the agentic method

- This project was built by **AI agents**, in a specific pattern the owner chose:



- A **workflow** = one planned batch of these agents run together (some in parallel), ending with a verify step. Work was split into **phases**, one batch each, so it stayed correct and controllable:

Phase	What it added	Agents
1	Multi-field core: models, 11 templates, scoring, providers, first UI	5
2	Optional AI scoring + reasoning (with fallback)	3
3	Shortlist, compare, export, history, shareable links	3
4	68 tests, input validation, error codes, docs	3
5	Stack Overflow source, required-filters, saved searches, free AI backends	4
6	The warm "woody" redesign	2

- Totals: **6 workflows · 20 sub-agents · 0 failures · ≈2.1M words of AI work**, then published to GitHub.
- **Why phased?** Each phase was small enough to verify fully before the next — the same reason human teams ship in small, tested increments.

13. Version control & publishing

- **Git** records every change as a "commit" (a labelled snapshot) — an undo history for the whole project.
- **GitHub** hosts that history online so others can see/clone it.
- What we did: created a `.gitignore` (to exclude `node_modules` , `.env` , generated files) → made the first commit → created the public repo `agent_max` → pushed.
- **Verified before publishing** that the `.env` (your token) was *not* included — only the blank `.env.example` went public.

14. Run it yourself — step by step

1. **Install Node.js** (v18+) from nodejs.org — this gives you `node` and `npm`.
2. Open a terminal in the project and go into it:

```
cd recruiter-ai
```

3. **Install the 2 libraries** the app needs:

```
npm install
```

4. **Start it:**

```
node server.js
```

5. Open <http://localhost:3000> in your browser. It fully works now — no keys needed.
6. (Optional) **Turn on free AI reasoning:**
 - Get a free key at console.groq.com.
 - Make a file named `.env` inside `recruiter-ai/` containing: `GROQ_API_KEY=your_key_here`
 - Restart (`node server.js`), tick the  **AI reasoning** box, search.
7. (Optional) Run the tests: `npm test` → expect `68 passing`.

15. Glossary (one line each)

- **API** — the set of requests a server accepts (its "menu").
- **Backend / server** — the program that does the work, off-screen.
- **Frontend / client** — the page you see and click, in your browser.
- **Node.js** — runs JavaScript on a server instead of a browser.
- **Express** — a Node library for building servers and APIs.
- **Endpoint / route** — one specific API address, e.g. `/api/search`.
- **REST** — an API style using URLs + verbs (GET = read, POST = send).
- **GET / POST** — "give me data" / "here's data, do something".
- **JSON** — plain-text format for structured data: `{ "key": "value" }`.
- **CSV** — data as comma-separated rows, like a spreadsheet export.
- **Provider** — a pluggable candidate data source (GitHub, etc.).
- **Criterion** — one weighted requirement in a job spec.
- **Weight** — how important a criterion is (1–5).
- **Raw score** — a single criterion's match, 0 to 1.
- **totalScore** — the final 0–100 number for a candidate.
- **Deterministic** — same input always gives same output (the rules engine).
- **LLM** — Large Language Model; the optional AI (Claude/Gemini/Llama).
- **Fallback** — a safe default used when the preferred path fails.
- **localStorage** — small storage inside your browser (no server).
- **Environment variable / .env** — a secret setting kept out of the code.
- **Git** — records every change to the code (version control).
- **GitHub** — a website that hosts Git projects.
- **Commit** — one saved snapshot of changes in Git.
- **.gitignore** — a list of files Git must never upload (e.g. `.env`).
- **Dependency / library** — outside code your project reuses (Express).

- **npm** — installs dependencies and runs tasks like `npm test` .
- **Agent / workflow** — an AI (or batch of AIs) that performs planned tasks.
- **Path-traversal** — an attack using `../` to read files it shouldn't; blocked here.

— End of guide —

Recruiter AI · github.com/Harbringer904/agent_max